Introduction to Python for Finance

Practice Exercises — print, input, if, for, while

Instructions: work through each exercise in order within a section. Write your code in a .py file or a notebook cell, run it, and check the output against what the exercise asks for.

Section 1 — print()

Goal: get comfortable displaying text, numbers, and formatted data on screen.

Exercise 1 — Hello, Analyst

Print the line: I am a Finance Analyst.

Then, on a new line, print your first name only.

> **Tip:** Each call to print() starts a new line automatically — you don't need to add \n between two separate print() calls.

Exercise 2 — Trading Desk Welcome

Print a 3-line welcome banner for a trading terminal, for example:

==========

Welcome to the Programming World!

==========

Exercise 3 — Quick Interest Calculation

Without using a variable, print the result of the expression 1000 * 0.05 directly inside print().

This represents 5% annual interest on a $1000 deposit.

> **Tip:** print() can evaluate an expression before displaying it — you don't always need to store the result in a variable first.

Exercise 4 — Printing Several Values at Once

Store a ticker symbol (e.g. "AAPL") and a price (e.g. 189.32) in two variables.

Print them on the same line separated by a space, then again separated by " | " using the sep argument.

> **Tip:** print(a, b, sep=" | ") controls what goes between arguments — useful for lining up financial data.

Exercise 5 — One-Line Portfolio Summary

Create variables for client name, portfolio value, and number of holdings.
Print a single multi-line summary using one print() call and \n inside the string.

Section 2 — input()

Goal: collect data from the user and understand why everything input() returns is text (str).

Exercise 6 — Greet the User

Ask the user to enter their name with input().
Print: Welcome to the Programming World, <name>!

Exercise 7 — input() Always Returns Text

Ask the user to enter a principal amount, e.g. 5000.
Print the value together with type(value) to show what Python thinks the type is.

> **Tip:** input() always returns a string, even if the user types numbers — you must convert it (e.g. float()) before doing math with it.

Exercise 8 — Sum Two Amounts

Ask the user for a loan amount and an interest amount (two separate input() calls).
Convert both to float and print their sum as the total amount due.

Exercise 9 — Simple Order Ticket

Ask for a company name (text) and a number of shares (integer, use int()).
Print an order description, e.g.: Order: BUY 50 shares of Tesla

> **Tip:** Use int(input(...)) when the value should be a whole number, like a share count — never a fraction of a share here.

Section 3 — if / elif / else

Goal: make decisions in code based on conditions.

Exercise 10 — Minimum Balance Check

Create a variable balance = 450.
If balance is below 500, print "Warning: below minimum balance." Otherwise print "Balance OK."

Exercise 11 — Profit, Loss, or Breakeven

Create a variable pnl (profit and loss), e.g. pnl = -120.5.

Print "Profit" if pnl > 0, "Loss" if pnl < 0, or "Breakeven" if pnl == 0.

> **Tip:** Comparing floats to exactly 0 can be risky in real applications due to rounding errors, but it's fine for this exercise.

Exercise 12 — Loan Approval by Credit Score

Create a variable credit_score.

Using if / elif / else, print:

"Approved — prime rate" if score >= 750

"Approved — standard rate" if 650 <= score < 750

"Declined" if score < 650

Exercise 13 — Transfer Fee Tiers

Create a variable amount for a money transfer.

Print the fee based on tiers: $0 fee if amount < 100, $2 fee if 100 <= amount < 1000, $5 fee if amount >= 1000.

Exercise 14 — Moving Average Signal

Create two variables: price and moving_average.

If price > moving_average print "BUY signal", if price < moving_average print "SELL signal", otherwise print "HOLD".

Exercise 15 — Simple Tax Bracket Calculator

Create a variable income.

Print the applicable bracket using elif, e.g.: 10% up to 10,000; 20% from 10,000 to 40,000; 30% above 40,000.

Print only the matching bracket rate, not the tax owed.

Section 4 — for loops

Goal: repeat an action over a known sequence.

Exercise 16 — Print Each Stock Price

Create a list prices = [120.5, 122.0, 119.8, 125.3].

Use a for loop to print each price on its own line.

Exercise 17 — Sum of Daily Returns

Create a list returns = [0.01, -0.02, 0.015, 0.005, -0.01].

Use a for loop and an accumulator variable to compute and print the total return.

Exercise 18 — Running Total of Expenses

Create a list expenses = [200, 450, 130, 90].

Use for i, e in enumerate(expenses) to print each expense with its position, and print the running total after each one.

Exercise 19 — Count Profitable Days

Create a list daily_pnl = [50, -20, 30, -10, 0, 75].

Use a for loop with an if inside it to count how many days had pnl > 0, and print the count at the end.

> **Tip:** Combine a counter (count = 0, then count += 1 inside the if) with the loop — this pattern appears constantly when screening financial data.

Section 5 — while loops

Goal: repeat an action until a condition is met, rather than a fixed number of times.

Exercise 20 — Compound Growth to a Target

Start with balance = 1000 and rate = 0.06.

Use a while loop that multiplies balance by (1 + rate) once per year until balance exceeds 2000.

Print the number of years it took.

Exercise 21 — Loan Amortization Countdown

Start with balance = 5000 and payment = 450.

Use a while loop that subtracts payment from balance each round and prints the remaining balance, stopping once balance <= 0.

Exercise 22 — Validate a Positive Amount

Use a while loop to repeatedly ask the user "Enter a deposit amount: " until they enter a number greater than 0.

Once valid, print "Accepted: " followed by the amount.

> **Tip:** Convert inside the loop with float(input(...)) and check the condition each time — this input-validation pattern is one of the most common uses of while.

Exercise 23 — Savings Goal Tracker

Start with savings = 0, monthly_deposit = 200, and goal = 3000.

Use a while loop to add monthly_deposit to savings each month and print the month number and current savings, stopping once savings >= goal.

Print the total number of months it took to reach the goal.